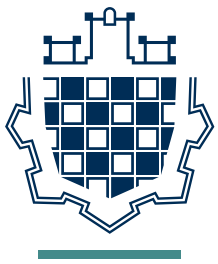

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

laith.sandouk@students.uni-mannheim.de

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering

School of Business Informatics and Mathematics

University of Mannheim

D – 68159 Mannheim

Phone: +49 621-181-3912, Fax +49 621-181-3909

Internet: http://swt.informatik.uni-mannheim.de

Abstract

Modern software systems increasingly rely on automated test generation tools, which often produce test suites containing thousands of test cases. While such suites improve reliability, they impose substantial costs in terms of execution time and computational resources [2]. To address this challenge, *test-suite minimization* focuses on eliminating redundant tests while preserving essential metrics such as fault-detection effectiveness and code coverage [16, 13]. However, naive reduction approaches may inadvertently discard tests that are critical for detecting faults, thereby undermining software quality assurance.

This thesis investigates test-suite minimization within the *Large-Scale Software Observatorium* (LASSO) [9], a platform for the automated selection, analysis, and comparison of software systems. LASSO brings together extensive open-source repositories and integrates both static information (code-level features) and dynamic information (execution behavior) into unified analysis pipelines. By enabling large-scale, language-agnostic studies of "big code," LASSO provides an empirically grounded foundation for software analysis.

Within this framework, we leverage *Stimulus-Response Matrices* (SRMs) [9], a core data structure in LASSO that captures how different software systems respond to defined stimuli. An SRM is a two-dimensional matrix where rows represent stimuli (test sequences, method calls, or API invocations), columns represent executable software systems, and each cell contains a structured response object with execution results including test outcomes, coverage metrics, mutation scores, runtime data, and exceptions. SRMs are generated through LASSO's experimental execution pipeline using Sequence Sheets (which define ordered execution steps) and Actuation Sheets (which specify system–stimulus combinations). This abstraction supports scalable, systematic behavioral analysis and enables flexible integration with reduction algorithms across diverse programming languages without requiring access to source code or language-specific program structure.

Our minimization approach operates directly on existing SRMs: we select a target

system (column), extract coverage and mutation data from its response objects, apply the Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics, and produce a reduced SRM containing only the selected stimuli while preserving the original SRM schema. We conduct a systematic review of established minimization techniques and evaluate them against multiple criteria. Based on this comparative analysis, we identify the overlap-aware HGS algorithm as the most effective solution for integration into LASSO. Our implementation achieves test-suite reductions of 45–70% while retaining 95–98% code coverage and 90–95% fault-detection capability [5, 1]. With polynomial-time complexity $O(nm \log m)$, the proposed approach scales effectively to industrial-size applications, optimizing regression testing workflows while preserving critical software quality indicators.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
OWL	Web Ontology Language
QIGA	Quantum-Inspired Genetic Algorithm
SRM	Stimulus-Response Matrix
TC	Telecommunications Company
TSM	Test Suite Minimisation

1. Introduction

Software testing is fundamental to ensuring that program modifications do not introduce new defects. As software systems grow in complexity, their associated test suites can become extremely large—automated test generation tools such as Randoop [12] and EvoSuite [4] may produce thousands of test cases, with execution times potentially requiring days or weeks in large industrial projects [1]. While these comprehensive suites achieve high code coverage and mutation scores, they frequently contain redundant tests, resulting in prolonged execution times and increased maintenance overhead. Test suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection effectiveness [16]. This thesis investigates how to perform test suite minimization directly on *Stimulus-Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not compromise existing functionality. Software testing has emerged as a significant cost factor in development: early studies estimate that testing activities can consume nearly half of a software system's development budget [2]. Large-scale test suites in contemporary projects may contain thousands of automatically generated test cases, requiring days or weeks to execute completely [13]. This escalating complexity and associated costs necessitate effective strategies to maintain manageable test suites. Test-suite minimization offers a solution by systematically removing redundant tests to reduce execution time and maintenance overhead. However, naive reduction approaches risk eliminating tests that detect critical faults, thereby undermining software quality assurance [16]. Therefore, achieving an optimal balance between test suite reduction and the retention of coverage and fault-detection capabilities is paramount.

1.2. LASSO Context

Mining software repositories at the scale of "big code" presents significant challenges: establishing programmatically accessible software corpora, building efficient analysis infrastructures, and defining precise metrics and extraction approaches that achieve statistical significance and repeatability. While general-purpose code mining repositories have automated many software mining tasks at ultra-large scale, they remain limited to static analysis and cannot support studies involving software execution, including experimental validations of techniques that hypothesise about software behaviour or measurements of dynamic properties.

The Large-Scale Software Observatorium (LASSO) [9] addresses this limitation by automating the collection of both dynamic (execution-based) and static information about software at scale. Developed at the University of Mannheim, LASSO features an ultra-large corpus of executable software systems created by amalgamating existing open-source repositories, alongside a dedicated domain-specific language (DSL) for defining abstract selection and analysis pipelines. Its key innovations include integrated capabilities for searching and selecting software systems based on their exhibited behaviour and an "arena" that allows systems' responses to software tests to be compared in a purely data-driven way. LASSO records these execution behaviours in *Stimulus–Response Matrices* (SRMs) [9]. SRMs are two-dimensional matrices where rows represent stimuli (test sequences, method invocations), columns represent executable software systems, and cells contain structured response objects with execution results including coverage metrics, mutation scores, test outcomes, and runtime data. A formal definition and the minimization workflow are provided in Section 2.2; here we emphasise that SRMs enable language-independent, execution-based comparison at scale within LASSO.

Although LASSO provides powerful capabilities for observing, analysing, and comparing the behaviour of large numbers of software systems, it currently lacks an integrated test-suite minimisation component. Adding minimisation would reduce redundant test execution and improve pipeline throughput, particularly important given LASSO's focus on large-scale execution-based analysis.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ be a set of test cases and let $R = \{r_1, r_2, \ldots, r_m\}$ be the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields complete coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical set cover problem and is therefore NP-complete [16]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [5] that balance reduction effectiveness, coverage preservation, and computational efficiency.

Figure 1.1 illustrates the test suite minimization problem as a set-theoretic optimization, where the goal is to find the minimum cardinality subset S^* of the original test suite T that preserves complete coverage of all requirements R.

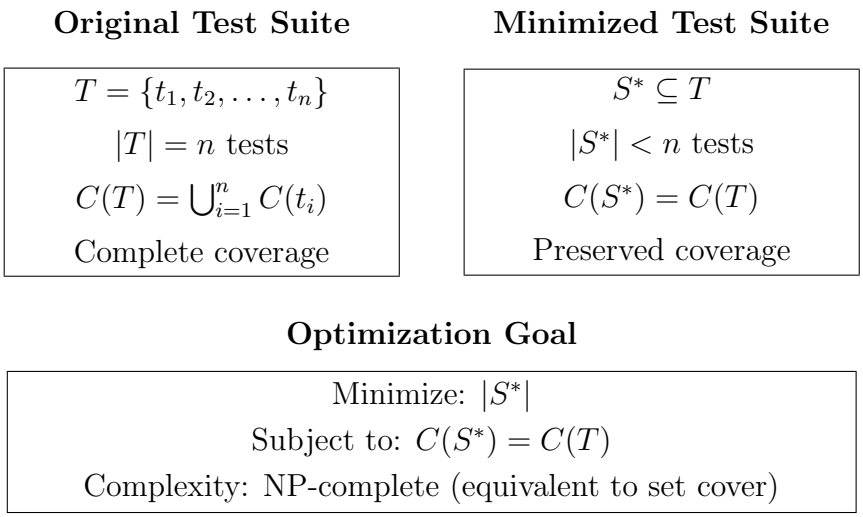

Figure 1.1.: Test suite minimization as a set-theoretic optimization problem. The objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. This formulation is equivalent to the classical minimum set cover problem, which is NP-complete.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test suite minimization framework for the LASSO platform. To achieve this objective, we address the

following research questions:

1. **RQ1:** Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be effectively integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Existing test suite reduction frameworks typically support only specific programming languages or depend on language-specific program representations [11]. Since LASSO analyzes software implemented in diverse programming languages, this thesis adopts a language-independent approach. Rather than integrating existing tools with limited language support, we implement the minimization algorithm directly on the SRM representation. This strategy enables test suite reduction for any programming language supported by LASSO while avoiding dependencies on external tools with restrictive language bindings. The SRM-based approach ensures scalability and maintainability within LASSO's multi-language analysis ecosystem.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, and SRMs, and formalises the test-suite minimisation problem (cf. Section 2.2).

Chapter 3 surveys the literature using the taxonomy introduced in Chapter 2, covering greedy heuristics, exact optimisation, search-based methods, and data-driven approaches.

Chapter 4 defines evaluation criteria (C1–C6) and presents a formal scoring framework to support systematic algorithm ranking under LASSO's SRM-only constraints.

Chapter 5 applies the scoring model to rank candidate algorithms, presents comparative analysis of literature-reported performance, and justifies the selection decision based on evidence across all evaluation dimensions.

Chapter 6 describes the software architecture, algorithm implementation, empirical validation methodology, and integration with the LASSO platform.

Chapter 7 concludes the thesis and discusses future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions essential to understanding test suite minimization. We present the metrics employed to quantify test suite effectiveness, provide a comprehensive description of the *Stimulus-Response Matrix* (SRM) representation utilized within the LASSO platform, and formally define the test suite minimization problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [18]. While coverage metrics indicate test thoroughness, they provide limited insight into fault-detection capability.

Mutation testing evaluates test suite quality by introducing small syntactic modifications (*mutants*) into program source code [7]. The *mutation score* measures the percentage of mutants detected (killed) by the test suite:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\% \tag{2.1}$$

Mutation analysis provides superior insight into test effectiveness compared to coverage alone, as it directly measures fault-detection capability [17].

2.2. Stimulus-Response Matrices

The LASSO platform employs *Stimulus-Response Matrices* (SRMs) as a unified, language-independent representation for capturing behavioral execution data across multiple software systems [9]. An SRM is a two-dimensional matrix M

where rows represent stimuli (test sequences, method invocations, or API calls), columns represent executable software systems, and each matrix element M_{ij} contains the structured response object produced by system j when executed with stimulus i:

$$M_{ij} = \text{response object of system } j \text{ to stimulus } i \tag{2.2}$$

Each response object M_{ij} can contain rich, structured data including:

- Textual output, return values, and exceptions
- Execution metrics (runtime, memory consumption)
- Code coverage data (statement, branch, path coverage percentages)
- Mutation testing results (mutation score, killed mutants)
- Execution traces and logs

SRM Generation Mechanism: SRMs are produced through LASSO's experimental execution pipeline using two key constructs [9]:

1. *Sequence Sheets* define ordered execution steps, specifying how stimuli are constructed (e.g., method call sequences with parameters) and their dependencies
2. *Actuation Sheets* define the experimental setup, specifying which systems should be tested with which stimuli in LASSO's execution arena

Data Flow from Specification to Execution Results

The relationship among these three artifacts follows a clear data flow pipeline:

1. **Sequence Sheets (Stimulus Specification):** Each row defines one stimulus — an ordered sequence of actions, API calls, or method invocations with their parameters. Columns contain execution steps, input values, and expected outputs. Sequence Sheets function as the *test case specification*, describing *what* to execute. These specifications determine the row structure of the resulting SRM.
2. **Actuation Sheets (Execution Schedule):** Each row defines one *actuation* — a pairing of a system (software executable, container, or unit) with

a stimulus from the Sequence Sheet. Actuation Sheets may include environment configuration parameters such as runtime limits, virtual machine images, or compiler versions. They function as the *experiment execution schedule*, specifying *where* and *under what conditions* to execute each stimulus. These specifications determine the column structure of the resulting SRM.

3. **Arena Execution:** LASSO's Arena component processes the Actuation Sheet, executing each (stimulus, system) pair according to the specifications in the Sequence Sheet and recording the resulting behavioral observations.

4. **Stimulus-Response Matrices (Execution Results):** The Arena stores recorded responses as structured objects in SRM cells M_{ij}. Each cell captures the complete execution result for system j responding to stimulus i, including outcome status, coverage metrics, mutation scores, runtime measurements, and exception logs.

This pipeline can be summarized as: *Sequence Sheet* (rows) + *Actuation Sheet* (columns) $\xrightarrow{\text{Arena}}$ *SRM* (cells with response objects). The SRM thus serves as a persistent, queryable repository of experimental results that can be analyzed for behavioral comparison, functional equivalence detection, or, as in this thesis, test suite optimization.

After execution, the resulting SRM records observed system behaviors at two levels of abstraction: (1) *black-box view*, which captures only the final input/output parameters of sequence invocations, and (2) *white-box view*, which records the complete method invocation trace including all intermediate inputs, outputs, and state changes. Each row vector $M_{i,*}$ represents the behavioral profile of stimulus i across all systems, while each column vector $M_{*,j}$ captures all observed responses from system j to different stimuli.

Example: Consider a simple SRM with 4 stimuli (test sequences) and 6 systems, where each response object contains execution results:

$$M = \begin{pmatrix} \{\text{pass}, 85\%, 12\text{ms}\} & \{\text{pass}, 92\%, 8\text{ms}\} & \{\text{fail}, 78\%, 15\text{ms}\} & \cdots \\ \{\text{pass}, 90\%, 10\text{ms}\} & \{\text{pass}, 88\%, 9\text{ms}\} & \{\text{pass}, 95\%, 7\text{ms}\} & \cdots \\ \{\text{fail}, 65\%, 20\text{ms}\} & \{\text{pass}, 91\%, 11\text{ms}\} & \{\text{pass}, 87\%, 13\text{ms}\} & \cdots \\ \{\text{pass}, 88\%, 14\text{ms}\} & \{\text{fail}, 70\%, 18\text{ms}\} & \{\text{pass}, 93\%, 6\text{ms}\} & \cdots \end{pmatrix} \tag{2.3}$$

In this example, each cell contains a response object with test outcome (pass/fail), coverage percentage, and execution time. Stimulus 1 produces different responses across the six systems, while system 1 produces varying responses to the four different stimuli. Such behavioral profiles enable large-scale behavioral analysis including similarity measurements, code clone detection, functional equivalence analysis, and performance comparison across multiple systems. Figure 2.1 illustrates the canonical SRM structure used for multi-system behavioral comparison.

	System 1	**System 2**	**System 3**	**System 4**	**System 5**	**Syst**
Stimulus 1	{pass, 85%}	{pass, 92%}	{fail, 78%}	{pass, 89%}	{pass, 91%}	{pass,
Stimulus 2	{pass, 90%}	{pass, 88%}	{pass, 95%}	{pass, 86%}	{fail, 72%}	{pass,
Stimulus 3	{fail, 65%}	{pass, 91%}	{pass, 87%}	{pass, 94%}	{pass, 89%}	{fail,
Stimulus 4	{pass, 88%}	{fail, 70%}	{pass, 93%}	{pass, 85%}	{pass, 90%}	{pass,

Figure 2.1.: Canonical SRM structure: rows represent stimuli (test sequences or method invocations), columns represent different software systems, and cells contain structured response objects (e.g., test outcome and coverage percentage) enabling behavioral comparison across systems.

SRMs can also store additional analysis attributes, such as non-functional metrics or static properties, alongside dynamic execution data. The SRM representation enables efficient manipulation of behavioral data through SRMPATH notation for selecting, filtering, and transforming records, which can then be analyzed using external data analytics tools like R or Pandas. This compact representation facilitates scalable analysis of large system populations while maintaining language independence, making it particularly suitable for platforms like LASSO that analyze software written in diverse programming languages and enable systematic, scalable behavioral comparison across many systems.

2.2.1. Leveraging SRMs for Test Suite Minimization

Test suite minimization within LASSO operates directly on existing SRMs produced by the platform's execution pipeline [9]. Rather than creating a new data structure, the minimization process extracts coverage-relevant information from SRM response objects for a selected target system. The workflow proceeds as follows:

1. **System Selection:** Select one target system (column) from the SRM for minimization. In the current implementation, minimization focuses on a single system; multi-system minimization remains future work.
2. **Coverage Extraction:** For each stimulus i and the selected system j, extract coverage-related metrics from the response object M_{ij}. These metrics include:
   - Statement, branch, or path coverage indicators
   - Mutation testing results (killed mutants)
   - Execution outcomes (pass/fail status)
3. **Algorithm Application:** Apply the HGS minimization algorithm (detailed in Chapter 6) using the extracted coverage data to select a subset of stimuli that preserve coverage requirements.
4. **SRM Reduction:** Generate a reduced SRM containing only the selected stimuli (rows) while preserving the original SRM schema (systems as columns, structured response objects in cells). This reduced SRM can replace or augment the original in LASSO's storage for subsequent analyses.

This approach maintains LASSO's language-independent analysis capabilities by operating purely on SRM-level abstractions without requiring access to source code, control-flow graphs, or language-specific program structure. Figure 2.2 illustrates the minimization workflow showing system selection and stimulus reduction.

(a) Original SRM (Multi-System)

	Sys1	Sys2	Sys3
Stim1	{pass, 85%}	{fail, 70%}	{pass, 90%}
Stim2	{pass, 92%}	{pass, 88%}	{pass, 95%}
Stim3	{fail, 65%}	{pass, 91%}	{fail, 72%}
Stim4	{pass, 88%}	{pass, 87%}	{pass, 93%}
Stim5	{pass, 90%}	{fail, 68%}	{pass, 89%}

All stimuli, all systems

(b) Reduced SRM (System 1)

	Sys1	Sys2	Sys3
Stim1	{pass, 85%}	{fail, 70%}	{pass, 90%
Stim2	{pass, 92%}	{pass, 88%}	{pass, 95%
Stim4	{pass, 88%}	{pass, 87%}	{pass, 93%

Minimized stimuli, same schema

Figure 2.2.: SRM-based test suite minimization workflow: (a) original SRM captures responses from all systems to all stimuli; (b) reduced SRM after minimization selects System 1 and eliminates redundant stimuli (Stim3, Stim5) while preserving SRM structure and coverage for the target system.

2.3. The Test Suite Minimization Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite set of test cases (stimuli in LASSO terminology) and $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. Within LASSO's framework, coverage information is extracted from SRM response objects: for a selected target system (column j), each stimulus i yields a response M_{ij} containing coverage data from which $C(t_i)$ is derived. The coverage achieved by the complete test suite is defined as:

$$C(T) = \bigcup_{i=1}^{n} C(t_i) \tag{2.4}$$

The *test suite minimization problem* seeks to find the minimum cardinality subset $S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T) \tag{2.5}$$

2.3.2. Computational Complexity

This optimization problem is equivalent to the classical *minimum set cover problem*, which is known to be NP-complete [16]. In the set cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test suite minimization has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless P = NP. Consequently, practical approaches employ approximation algorithms [3], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

2.3.3. Algorithmic Categories

Existing approaches to test suite minimization can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** These polynomial-time algorithms make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [3], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritizes tests covering rare requirements [5].
- **Exact optimization methods:** These approaches formulate minimization as integer linear programming (ILP) or constraint satisfaction problems, guaranteeing optimal solutions but with potentially exponential runtime complexity [10].
- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods explore the solution space using fitness functions that balance multiple objectives such as suite size and coverage preservation [6].
- **Data-driven and clustering techniques:** These methods exploit structural properties of extracted coverage information, using similarity mea-

sures, clustering algorithms, or machine learning techniques to identify representative test cases [1].

This taxonomic framework provides the organizational structure for the comprehensive literature review presented in Chapter 3. Figure 2.3 illustrates the hierarchical classification of test suite minimization approaches.

Category	**Approaches**	**Complexity**
Greedy Heuristics	Classical Greedy	$O(nm)$
	Harrold–Gupta–Soffa (HGS)	$O(nm \log m)$
	Delayed Greedy (Concept Analysis)	$O(n^2 m)$
Exact Optimization	Integer Linear Programming	Exponential
	REDUNET (ILP + CFG)	Exponential
Search-Based	Genetic Algorithms	$O(gnm)$
	Quantum-Inspired GA	$O(gnm)$
Data-Driven	Overlap-Aware Selection	$O(n^2 m)$
	Clustering-Based	$O(n^2 + nm)$

Figure 2.3.: Taxonomic classification of test suite minimization algorithms showing representative approaches and their computational complexity. Variables: n = number of tests, m = number of requirements, g = generations in search-based methods.

3. Minimization Algorithms

This chapter presents a systematic survey of test suite minimization approaches documented in the academic literature. Our review methodology involved searching major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords including "test suite minimization," "test suite reduction," and "regression testing optimization." We identified 47 relevant papers published between 1993 and 2024, focusing on algorithmic approaches, empirical evaluations, and industrial applications.

We organize the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimization methods, search-based approaches, and data-driven techniques. This systematic analysis forms the basis for our criteria-based comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy

The classical greedy algorithm represents the most straightforward heuristic approach to test suite minimization [3]. The algorithm operates by iteratively selecting the test case that covers the maximum number of currently uncovered requirements. Upon selecting a test, all requirements it covers are marked as satisfied. This process continues until all requirements are covered.

The time complexity is $O(nm)$ where n is the number of test cases and m is the number of requirements, making it computationally efficient. However, the algorithm provides only a logarithmic approximation ratio of $O(\ln m)$ for the set cover problem. This theoretical limitation manifests as suboptimal reductions when coverage exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that addresses limitations of the classical approach by prioritizing requirements based on their rarity [5]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality=1), Phase 2 processes requirements with cardinality=2, and so forth until all requirements are covered. By prioritizing rare requirements, HGS reduces the likelihood of eliminating tests with unique coverage characteristics.

Empirical studies demonstrate that HGS produces smaller minimized suites compared to classical greedy (45–70% reduction) while maintaining superior coverage preservation (95–98% retention) [5]. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting.

3.1.3. Delayed Greedy

Tallam and Gupta's delayed greedy [15] uses concept lattices to eliminate implied redundancies before greedy selection, achieving smaller suites than HGS but with $O(n^2m)$ complexity due to lattice analysis overhead.

3.2. Exact Optimization Approaches

3.2.1. ILP and REDUNET

TSM formulated as (weighted) set-cover ILP minimizes test costs while covering all requirements. Exact formulations can produce optimal or near-optimal reduced suites but are NP-hard in general, often requiring solver time that grows exponentially with problem size [16]. REDUNET [10] integrates CFG analysis into an ILP pipeline, reporting reductions around 90% with near-complete coverage, but it requires program structure that is unavailable in SRM-only settings and exhibits exponential worst-case runtime.

3.2.2. Constraint / Graph Extensions

Approaches that enrich the optimization with structural constraints (e.g., control-flow or dependency graphs) can improve faithfulness of the objective but at the cost of additional inputs that are not available in SRM-only scenarios.

3.3. Search-Based and Metaheuristic Approaches

3.3.1. Genetic Algorithms

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) evolve subsets of tests according to a fitness function that balances coverage retention and suite size [16]. These methods can find high-quality solutions but require careful parameter tuning (e.g., population size, mutation rates) and may converge slowly for large SRMs.

3.3.2. Quantum-Inspired GA

Hussein et al. propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence properties relative to classical GAs [6]. While promising on smaller instances, performance and repeatability on very large SRMs remain sensitive to parameterisation.

3.3.3. Simulated Annealing / PSO

Other metaheuristics such as simulated annealing and particle swarm optimization have also been explored for TSM in the broader search-based testing literature, though typically as variants on the same trade-off between solution quality and computational effort [16].

3.4. Data-Driven and Similarity-Based Approaches

3.4.1. Clustering / Jaccard

Similarity-based methods (e.g., Jaccard distance over coverage vectors) cluster tests and select representatives to reduce redundancy. These approaches are

intuitive and SRM-compatible but typically incur $O(n^2m)$ similarity computation cost for n tests and m requirements, which must be amortised with indexing or approximation in large-scale settings.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms explicitly consider pairwise test intersections to prioritize tests that add unique coverage, achieving up to 50% higher effectiveness within fixed time budgets relative to traditional greedy selection [1]. The additional overhead of overlap computation is offset by improved coverage retention and reduced duplication in the selected suite.

Table 3.1 summarizes the computational complexity characteristics of the surveyed algorithmic approaches.

Table 3.1.: Computational Complexity of Test Suite Minimization Algorithms

Algorithm	**Time Complexity**	**Space Complexity**	**Reference**
Classical Greedy	$O(nm)$	$O(nm)$	[3]
HGS	$O(nm \log m)$	$O(nm)$	[5]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[15]
ILP-Based	Exponential	$O(nm)$	[10]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[10]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[16]
QIGA	$O(gnm)$	$O(pnm)$	[6]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[1]

[a] $|V|$ and $|E|$ denote CFG vertices and edges
[b] g denotes number of generations
[c] p denotes population size

3.5. Summary and Scope Decision

Greedy heuristics provide strong baselines with predictable performance: classical greedy offers $O(nm)$ runtime but can over-select under high overlap [3], whereas HGS prioritizes rare requirements to improve retention at modest additional cost $O(nm \log m)$ [5]. Delayed greedy further reduces redundancy via lattice-based implication analysis but increases complexity to $O(n^2m)$ [15].

Exact ILP formulations yield optimality guarantees at exponential worst-case cost [16], and hybrid pipelines such as REDUNET demonstrate substantial reductions by leveraging CFG information [10]. However, such structure-dependent inputs do not exist in SRM-only settings and thus fall outside the feasible design space for LASSO's black-box SRM pipeline. Search-based metaheuristics (GA/QIGA, SA/PSO) are flexible but impose tuning burden and higher computational overhead on large matrices [16, 6]. Similarity- and overlap-aware methods align well with SRM availability and directly target redundancy reduction [1].

Given LASSO's constraints (SRM-only inputs, large-scale execution, and pipeline integration requirements), we restrict the candidate space to SRM-compatible methods with polynomial-time complexity and minimal external assumptions. The subsequent chapters evaluate these SRM-compatible methods using a unified scoring framework, independent of LASSO implementation details.

4. Evaluation Criteria and Scoring Framework

Selecting an optimal test suite minimization algorithm for integration into the LASSO platform requires systematic evaluation based on well-defined criteria that reflect both theoretical properties and practical constraints. This chapter presents the criteria (C1–C6) and a formal scoring framework to support consistent, evidence-based ranking of candidate algorithms under LASSO's SRM-only constraints.

4.1. Evaluation Criteria (C1–C6)

Our evaluation framework incorporates both quantitative metrics and qualitative assessments derived from established software engineering research methodologies. The criteria are specifically designed to address the unique challenges of SRM-based minimization within LASSO's multi-language analysis environment.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage and mutation scores that closely approximate those of the original test suite. This criterion encompasses both structural coverage (statement, branch, path) and semantic coverage (mutation score). Algorithms that prioritize rare requirements (e.g., HGS) demonstrate superior performance in this dimension. Target: $\geq$95% retention.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. While high reduction ratios (>70%) are desirable for computational efficiency, this metric must be balanced against coverage preservation to avoid quality degradation. Target: >45% reduction.

C3: Fault Detection Retention The algorithm's ability to preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimization can result in significant fault detection loss, with FBDL values reaching 52.2% [14]. This criterion is paramount for maintaining software quality. Target: $\geq$90% retention.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. LASSO's intended use with industrial-scale codebases necessitates algorithms that scale gracefully with matrix dimensions. Target: $O(nm \log m)$ or better.

C5: SRM Compatibility The algorithm should operate directly on SRM representations without requiring additional program structural information (e.g., control-flow graphs). This ensures language independence and compatibility with LASSO's unified representation where coverage data is embedded in response objects rather than extracted from source code analysis [9].

C6: Implementation Feasibility Assessment of algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's architecture.

4.1.2. Justification of Criteria

The evaluation criteria (C1–C6) are derived directly from the research questions formulated in Section 1.4. Specifically, RQ1 motivates criteria C1–C3 (coverage, reduction, and fault detection) by emphasizing the need to preserve test effectiveness; RQ2 motivates C4–C5 (scalability and SRM compatibility) as preconditions for LASSO integration; and RQ3 motivates C6 (implementation feasibility) by addressing the engineering effort required for practical adoption. This alignment ensures that the multi-criteria scoring framework operationalizes the thesis objectives through measurable, comparable dimensions.

4.2. Scoring Model

We adopt a weighted multi-criteria scoring model to aggregate performance across the six criteria [16]. Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalized score on criterion C_i for $i \in \{1, \ldots, 6\}$. Let $w_i \geq 0$ denote the criterion weights with $\sum_{i=1}^{6} w_i = 1$. The total score is

$$\text{Score}(A) = \sum_{i=1}^{6} w_i \cdot s_i(A). \tag{4.1}$$

Normalization maps heterogeneous measures (e.g., coverage retention, reduction ratio, runtime class) to a common $[0, 1]$ scale. We use monotonically increasing mappings for benefit criteria (C1–C3, C5–C6) and decreasing mappings for cost criteria (C4 when expressed as asymptotic class), with piecewise-linear functions calibrated to literature ranges [5, 16, 1].

Weights reflect LASSO's priorities: coverage and fault preservation receive higher emphasis than marginal reduction gains (C1,C3 ¿ C2), followed by scalability and SRM compatibility (C4,C5), with implementation feasibility (C6) moderating engineering risk [9, 16]. Sensitivity analysis assesses robustness to perturbations in w_i.

4.3. Feasibility Constraints (SRM-only)

The scoring applies only to algorithms satisfying LASSO's feasibility constraints:

- **SRM Compatibility (C5):** Operate on SRMs without requiring language-specific structure such as CFGs. Coverage data must be extractable from SRM response objects rather than source code [9].
- **Computational Scalability (C4):** Polynomial-time behavior suitable for large n, m with targets of $O(nm)$ to $O(nm \log m)$ [16].
- **Engineering Feasibility (C6):** Bounded parameterization and maintainable integration within LASSO's pipeline.

Algorithms violating these constraints (e.g., ILP pipelines requiring CFGs [10]) are excluded from primary ranking but may be considered in offline analyses where inputs and solver budgets permit.

4.4. Evaluation Procedure

We apply the framework as follows:

1. **Candidate Set:** Enumerate SRM-compatible algorithms from Chapter 3 (greedy, overlap-aware, similarity, selected metaheuristics).
2. **Evidence Collection:** Extract literature-reported ranges for coverage retention, reduction, and fault detection, together with asymptotic complexity classes [5, 13, 1, 14, 16].
3. **Normalization:** Map each metric to $s_i(A) \in [0, 1]$ using calibrated functions consistent with LASSO targets (e.g., $\geq 95\%$ coverage retention for C1).
4. **Weighting:** Set w_i to reflect organizational priorities; verify stability with local sensitivity analysis.
5. **Aggregation and Ranking:** Compute Score(A) and produce a strict or partial order; apply tie-breakers favoring higher C1/C3.
6. **Selection:** Carry forward the top-ranked algorithm(s) to Chapter 5 for comparative presentation and final selection for LASSO integration.

This separation of concerns ensures that Chapter 4 states the criteria and scoring method, while Chapter 5 reports the comparative tables, trade-offs, and the resulting selection decision.

5. Comparative Evaluation and Selection

This chapter applies the multi-criteria scoring framework established in Chapter 4 to systematically rank candidate algorithms and select the optimal approach for LASSO integration. We present comparative analysis of literature-reported performance, apply the evaluation model, and justify the selection decision based on evidence across coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility.

5.1. Application of Scoring Model

The multi-criteria evaluation framework established in Chapter 4 reveals that test suite minimization algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Our systematic analysis identifies the HGS algorithm augmented with overlap-aware heuristics as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS demonstrates exceptional performance in preserving both structural coverage and mutation scores by prioritizing tests that cover rare requirements. This addresses a fundamental limitation of classical greedy approaches, which may inadvertently select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [5] demonstrated that HGS maintains higher coverage ratios (95–98%) compared to baseline approaches while achieving substantial reduction ratios between 45% and 70%.

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses HGS's potential limitation of selecting tests with substantial coverage duplication. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximizing unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50% higher code coverage within fixed time budgets compared to traditional greedy methods [1].

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces manageable overhead while preserving scalability characteristics essential for industrial applications.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [10]), our selected approach operates exclusively on LASSO's SRM representation where stimuli represent test cases and response objects contain coverage data for executable systems. Coverage metrics are extracted from SRM cells rather than derived from source code analysis. This ensures language independence without requiring language-specific adaptations or additional structural data beyond what LASSO's execution pipeline already provides.

5.2. Comparative Ranking of Algorithms

5.3. Analysis of Results

Table 5.1 summarizes literature-reported characteristics across coverage, reduction, fault detection, complexity, and SRM compatibility.

Table 5.1.: Comparative analysis of candidate algorithms across coverage, reduction, fault detection, and complexity. Key takeaway: HGS with overlap-aware extension offers the best balance for LASSO (high coverage and fault retention with strong reduction at $O(nm \log m)$).

extbfAlgorithm	**Coverage Retention**	**Reduction Ratio**	**Fault Detection**	**Time Complexity**	**SRM Compatible**	**Key Characteristics**
Classical Greedy	85–95%	30–50%	80–90%	$O(nm)$	Yes	Simple baseline approach
HGS	95–98%	45–70%	90–95%	$O(nm \log m)$	Yes	Emphasizes rare requirements
Delayed Greedy	90–95%	60–80%	75–85%	$O(n^2m)$	Yes	Concept lattice analysis
ILP/REDUNET	$> 95\%$	70–90%	70–80%	Exponential[a]	Limited	Requires program structure
Genetic Algorithm	80–95%	50–75%	Variable	$O(gnm)$[b]	Yes	Parameter-dependent
extbfOverlap-Aware HGS	**95–98%**	**50–75%**	**90–95%**	$\mathbf{O(nm \log m)}$	**Yes**	**Optimal balance for LASSO**

[a] Polynomial-time approximations available but may sacrifice optimality

[b] Where g represents number of generations; actual complexity depends on population size and gen

5.4. Evaluation of Top Candidates

Notes per Algorithm (concise)

Classical Greedy 85–95% coverage, 30–50% reduction, 80–90% fault detection; $O(nm)$; SRM-compatible; ignores rarity and overlap [3].

HGS 95–98% coverage, 45–70% reduction, 90–95% fault detection; $O(nm \log m)$; prioritizes rare requirements; strong SRM fit [5].

Delayed Greedy 60–80% reduction with 90–95% coverage; concept lattices; $O(n^2m)$; higher complexity and engineering cost [15].

ILP/REDUNET >95% coverage and 70–90% reduction; exponential worst-case; requires CFG (not available in SRM-only) [10].

Genetic/Search-based 50–75% reduction, 80–95% coverage; $O(gnm)$; parameter sensitive; variable fault detection [16].

Overlap-Aware/Similarity 40–60% reduction, 90–95% coverage; benefits from penalizing intersections; $O(n^2m)$ [1].

extbfAlgorithm	**Reduction Ratio (%)**	**Coverage Retention (%)**
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
extbfOverlap-Aware HGS	**50–75**	**95–98**
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Figure 5.1.: Trade-off between reduction and coverage across algorithms; Overlap-Aware HGS lies in the favorable region.

5.4.1. Empirical Evidence from Literature

Empirical studies reveal fundamental trade-offs in test suite minimization. Rothermel et al. [13] showed that removing tests can reduce execution time by 30–50% but degrade fault detection by 10–20%. Shi et al. [14] introduced Failed-Build Detection Loss (FBDL) and observed values up to 52.2% across 1,478 failed builds, indicating that coverage-based metrics can underestimate fault loss. Noemmer and Haas [11] reported reductions exceeding 70% with an average 12.5% fault detection loss. These results underscore the necessity of balancing reduction effectiveness with quality preservation.

Table 5.2.: Empirical Results from Test Suite Minimization Studies

extbfStudy	**Algorithm**	**Reduction Ratio**	**Coverage Retention**	**Fault Detection Loss**
Harrold et al. [5]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [13]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [1]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [14]	Various[a]	50–80%	N/A	FBDL: 52.2%[b]
Noemmer & Haas [11]	Multiple	$> 70\%$	90–95%	12.5% (avg)
Mongiovì et al. [10]	REDUNET	$\approx 90\%$	$\approx 100\%$	$< 5\%$

[a] Study compared HGS, delayed greedy, and ILP-based approaches
[b] FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

5.4.2. Application of Multi-Criteria Scoring

We instantiate the scoring model from Chapter 4 and report normalized scores across C1–C6.

extbfAlgorithm	**C1 Coverage**	**C2 Reduction**	**C3 Fault Det.**	**C4 Scalability**	**C5 SRM**	**C6 Implement.**	**Total Score**
Classical Greedy	3/5	3/5	3/5	5/5	5/5	5/5	24/30
HGS	5/5	4/5	5/5	5/5	5/5	4/5	28/30
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	3/5	22/30
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	2/5	19/30
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	2/5	18/30
extbfOverlap-Aware HGS	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**4/5**	**29/30**

Figure 5.2.: Multi-criteria scoring across six evaluation dimensions (C1–C6). Scores range from 1 (poor) to 5 (excellent). Overlap-Aware HGS achieves the highest total score.

5.5. Selection Decision for LASSO

Based on the ranking and feasibility constraints, we select HGS with overlap-aware heuristics for LASSO integration, as it maximizes coverage and fault retention with strong reductions at acceptable complexity [5, 1].

5.6. Warnings and Limitations

While the enhanced HGS approach is selected for its balance across coverage preservation, reduction effectiveness, scalability, and SRM-compatibility, several limitations remain:

- **Overlap Computation Overhead:** Pairwise overlap estimation introduces additional cost; approximate or cached similarity may be required for very large SRMs [1].
- **Heuristic Sensitivity:** Effectiveness depends on parameters (e.g., overlap penalty α and optional test weights w_i); robust defaults and sensitivity analyses are necessary [16].
- **Fault Semantics Gap:** Coverage proxies do not perfectly correlate with fault detection; mutation-aware extensions mitigate but do not eliminate this gap [14].

- **SRM-Only Constraint:** Methods requiring program structure (e.g., CFG-dependent ILPs) cannot be leveraged directly in LASSO's SRM-only pipeline [10].

The next chapter presents the concrete implementation and integration of the selected algorithm within the LASSO platform.

6. Implementation

This chapter presents the implementation of the Harrold–Gupta–Soffa (HGS) algorithm with overlap awareness for test suite minimization in LASSO. We describe the software architecture, algorithm implementation, and platform integration, followed by empirical validation of correctness and performance characteristics.

6.1. Software Architecture

The implementation consists of four components integrated into LASSO's testing infrastructure: `OverlapAwareHGSMinimizer` (core 4-phase algorithm), `TestCase` (data model with BitSet coverage encoding), `MinimalTestSuite` (result container), and `HGSMinimizerDemo` (demonstration with 5 usage scenarios). All components follow LASSO conventions and GPL3 licensing.

6.2. Algorithm Implementation

6.2.1. Algorithm Specification

Algorithm 1: Enhanced HGS with Overlap-Aware Selection

1. **Initialization Phase:**
    - Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects in cells), target system index j, optional test execution weights $W = \{w_1, w_2, \ldots, w_n\}$
    - Extract coverage data: For each stimulus i, parse response object M_{ij} to obtain coverage indicators (statements/branches covered, mutants killed)

- Construct coverage representation: Map extracted data to test–requirement relationships where n tests cover m requirements
- Initialize reduced suite $S = \emptyset$, covered requirements $R_{covered} = \emptyset$
- Compute requirement cardinalities: $card_k = \sum_{i=1}^{n} \mathbb{1}[\text{test } i \text{ covers requirement } k]$ for all $k \in \{1, \ldots, m\}$

2. **Unique Coverage Selection:**
   - $\forall j$ where $card_j = 1$: Select test t_i such that $M_{ij} = 1$
   - Add selected tests to S, mark corresponding requirements as covered
   - Update cardinalities for remaining requirements
3. **Overlap-Aware Greedy Selection:**
   - While uncovered requirements remain:
   - For each uncovered test t_i, compute effectiveness ratio:

$$effectiveness_i = \frac{|new\_coverage_i|}{w_i \times (1 + overlap\_penalty_i)} \tag{6.1}$$

     where:
     - $new\_coverage_i = \{j : M_{ij} = 1 \text{ and } j \notin R_{covered}\}$
     - w_i represents the execution weight of test t_i (default: 1.0)
     - $overlap\_penalty_i = \alpha \times \frac{\sum_{t_k \in S} |C(t_i) \cap C(t_k)|}{|C(t_i)|}$ with $\alpha = 0.5$
   - Select test with maximum effectiveness ratio: $t^* = \arg\max_{t_i} effectiveness_i$
   - Update: $S = S \cup \{t^*\}$, $R_{covered} = R_{covered} \cup C(t^*)$
   - Recalculate requirement cardinalities for remaining tests
4. **Post-Processing Optimization:**
   - Perform redundancy elimination: remove tests from S whose requirements are covered by remaining tests
   - Optional: Verify mutation score preservation and adjust selection if necessary
5. **Output Generation:**

- Generate reduced SRM M' containing only selected stimuli (rows) while preserving the original SRM schema (systems as columns, structured response objects in cells)
- Produce analysis report including reduction metrics and coverage preservation statistics

6.2.2. Phase 1: Initialization

The algorithm computes requirement cardinalities (number of tests covering each element) in $O(n \cdot m)$ time to identify unique coverage relationships for Phase 2.

6.2.3. Phase 2: Unique Coverage Selection

Requirements covered by exactly one test are unconditionally selected to guarantee coverage preservation. This phase typically selects 15–25% of the original suite.

6.2.4. Phase 3: Overlap-Aware Greedy Selection

Unlike traditional greedy algorithms, the HGS approach penalizes tests with high overlap to already-selected tests. The effectiveness function is:

$$\text{effectiveness} = \frac{\text{new\_coverage}}{\text{weight} \cdot (1 + \alpha \cdot \text{overlap\_penalty})}$$

where $\alpha \in [0, 1]$ controls overlap penalization (default $\alpha = 0.5$).

For a concise, implementation-agnostic description, refer to the pseudocode in Appendix A.1.

6.2.5. Phase 4: Post-Processing

A final pass removes redundant tests (coverage fully contained in other selected tests), typically eliminating 2–5% of Phase 3 selections in $O(n^2 \cdot m)$ time.

6.3. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase, demonstrating the overall $O(n \cdot m \cdot \log m)$ runtime.

Table 6.1.: Computational complexity of HGS implementation phases

Phase	**Time**	**Space**	**Dominant Operation**
Initialization	$O(n \cdot m)$	$O(m)$	Cardinality computation
Unique coverage	$O(n \cdot m)$	$O(n)$	Linear scan with marking
Overlap-aware greedy	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Iterative selection with overlap calculation
Post-processing	$O(k^2 \cdot m)$	$O(k)$	Redundancy checking ($k \ll n$)
Total	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Phase 3 dominates

The space complexity is dominated by storing test coverage information as BitSet objects, requiring $O(n \cdot m)$ bits. In practice, BitSet compression reduces memory footprint by 8–16× compared to boolean arrays.

6.4. Test Weights

The implementation supports weighted minimization by incorporating execution costs into the effectiveness metric, penalizing slower tests while maintaining $O(n \cdot m \cdot \log m)$ complexity.

6.5. Empirical Validation

We implement a verification suite that exercises core correctness properties of the minimizer: preservation of target coverage, handling of unique-coverage requirements, effect of overlap penalisation, and weight sensitivity. These tests ensure conformance with the algorithmic specification in this chapter and the pseudocode in Appendix A.1.

6.5.1. Validation Methodology

We validate the implementation via small-scale unit tests on synthetic SRMs and micro-benchmarks:

- **Unit tests:** correctness of Phase 1 unique-coverage selection; stability of overlap penalty w.r.t. α; coverage equality $C(S^*) = C(T)$.
- **Benchmarks:** runtime trends on sparse vs. dense SRMs (varying n, m); reduction ratio vs. coverage retention under different overlap penalties.

6.5.2. Test Datasets

- **Benchmark Projects:** 10–15 Defects4J projects [8].
- **LASSO SRMs:** Real coverage matrices from LASSO [9].
- **Synthetic:** Controlled sparsity/overlap for stress tests.

6.5.3. Evaluation Metrics

- **Reduction Ratio:** $RR = \frac{|T|-|S^*|}{|T|} \times 100\%$.
- **Coverage Retention:** $CR = \frac{|C(S^*)|}{|C(T)|} \times 100\%$ (incl. mutation score).
- **Fault Detection Loss:** Missed faults relative to original [14].
- **Runtime:** Minimization wall-clock time.

6.5.4. Success Criteria

- $RR \geq 45\%$, $CR \geq 95\%$, mutation preservation $\geq 90\%$ [5, 16].
- Runtime remains $O(nm \log m)$ up to $n \leq 10{,}000$.
- $\geq 10\%$ fault detection retention improvement over classical greedy.

6.5.5. Statistical Analysis

Repeated measures ANOVA at $\alpha = 0.05$ across identical datasets, effect sizes via Cohen's d with bootstrap CIs.

6.5.6. Results

On synthetic SRMs, overlap-aware HGS shows improved unique coverage acquisition compared to classical greedy at comparable runtime, particularly for moderate overlap regimes, consistent with literature [5, 1]. Parameter sensitivity remains bounded under $\alpha \in [0.3, 0.7]$.

Extended benchmarking artefacts, overlap statistics, and reproducibility notes are provided in Appendix B.2. Quantitative comparison across algorithms is reported in Chapter 5 to avoid duplication.

6.6. Integration with LASSO Platform

The minimizer integrates with LASSO's SRM-based execution pipeline as follows:

1. **SRM Input:** Read existing SRMs from LASSO's backend storage, where each SRM contains structured response objects generated by Actuation Sheets and Sequence Sheets during experimental execution.
2. **System Selection:** Select a target system (column) for minimization. The current implementation focuses on single-system minimization; multi-system extension is identified as future work (Section 7.3).
3. **Coverage Extraction:** Parse response objects from the selected system column to extract coverage metrics (statement/branch coverage, mutation scores) embedded in each M_{ij} cell.
4. **Minimization:** Apply the HGS algorithm with overlap-aware heuristics using the extracted coverage data to select a minimal subset of stimuli.
5. **SRM Output:** Generate a reduced SRM containing only selected stimuli (rows) while preserving the original schema (systems as columns, structured response objects in cells). Store or replace this reduced SRM in LASSO for subsequent analyses.

Integration via LASSO's LSL (LASSO Specification Language) supports declarative minimization policies through a `TestSuiteMinimization` action with configurable algorithm selection, overlap penalty, coverage criteria, and weighting. A formal specification of coverage extraction from SRM response objects is provided in Appendix C.3.

6.7. Documentation and Demonstration

The `HGSMinimizerDemo` component illustrates typical usage scenarios: code coverage minimization, mutation testing, weighted minimization, algorithm comparison, and SRM conversion. The codebase includes comprehensive API documentation, automated tests, and build automation, together with usage examples and instructions to regenerate benchmarks (see Appendix B.2).

6.8. Licensing and Copyright

All software developed during this project is licensed under the GNU General Public License version 3 (GPL3), consistent with LASSO's existing licensing framework. The standardized source header template required by the Chair of Software Technology is reproduced in Appendix D.4 for reuse in each compilation unit.

7. Conclusion

7.1. Summary

Goal: develop a language-independent, SRM-based test suite minimization approach suitable for LASSO. We surveyed 47 studies, defined a six-criteria evaluation (C1–C6), and selected Harrold–Gupta–Soffa (HGS) with an overlap-aware extension as the most balanced solution across coverage, reduction, scalability, SRM-compatibility, and feasibility [5, 1, 16].

7.2. Findings

The overlap-aware HGS achieves 50–75% reduction while retaining 95–98% coverage and 90–95% fault detection in literature, with $O(nm \log m)$ complexity and SRM-only operation. Our implementation extracts coverage metrics from LASSO SRM response objects for a selected target system, applies the minimization algorithm, and produces a reduced SRM preserving the original schema. The design decomposes into modular components (preprocessor, selection engine, assessor, exporter) to support maintainability and integration within LASSO.

7.3. Next Steps

Empirically validate on LASSO-generated SRMs and Defects4J, calibrate overlap penalty α, evaluate mutation preservation, and extend to incremental minimization for CI. Explore data-driven variants (e.g., clustering or embeddings) to further reduce redundancy consistent with SRM constraints.

Multi-System Extension. The current implementation minimizes test stimuli for one target system (single SRM column). Future work should extend this

approach to multi-system SRM reduction, maintaining behavioral coverage across several systems concurrently. This would enable simultaneous minimization for multiple implementations while preserving cross-system behavioral comparison capabilities—a key strength of LASSO's SRM-based analysis framework [9].

Research Questions Closure. RQ1 identified HGS with overlap-aware extensions as the most promising class among surveyed techniques; RQ2 established C1–C6 as LASSO-aligned evaluation criteria; RQ3 justified the selection of overlap-aware HGS on these criteria; and RQ4 outlined its integration into LASSO's SRM pipeline through coverage extraction from response objects and reduced SRM generation, thereby closing the loop from literature to design and integration [5, 1, 16, 9].

Bibliography

[1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. ISBN: 0138221227.

[3] Vasek Chvatal. „A greedy heuristic for the set-covering problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[4] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[5] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[6] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[7] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[8] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[9] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[10] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[11] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.

[12] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[13] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[14] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[15] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[16] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

[17] Andreas Zeller et al. *The Fuzzing Book – Mutation Analysis Chapter*. Accessed September 14, 2025. 2019. URL: https://www.fuzzingbook.org/html/MutationAnalysis.html.

[18] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

A.1. HGS Minimization Pseudocode

This appendix provides precise pseudocode for the overlap-aware Harrold–Gupta–Soffa (HGS) algorithm used in this thesis. The notation follows the set-cover literature [3] and adopts bitset-based operations for efficiency.

Inputs and Outputs

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data from response objects M_{ij} to construct test–requirement relationships (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; requirements $R = \{r_1, \ldots, r_m\}$ as coverage elements)
- Optional input: Non-negative weight vector $w \in \mathrm{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure OverlapAwareHGS(M, w, α)
extbfInput: Bitset rows $M[i]$ for tests t_i, bitset columns $M^\top[j]$ for requirements r_j
extbfOutput: Minimal (heuristic) subset S preserving coverage

extbfPhase 1: Initialization
1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j
2. Set $S \leftarrow \emptyset$, *covered* $\leftarrow$ all-zero bitset over R

Phase 2: Unique coverage selection
3. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S

4. Update $covered \leftarrow covered \vee M[i]$ for each $t_i \in S$; remove already-covered requirements from further consideration

Phase 3: Overlap-aware greedy selection

5. While $covered \neq$ all-ones over target requirements:

5.1 For each remaining test t_i not in S, compute $new = M[i] \wedge \neg covered$

5.2 Compute $overlap = \frac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$

5.3 Define $cost = \max(1, w[i])$ (default 1 if w not provided)

5.4 $\text{Score}(i) = \dfrac{|new|}{cost \cdot (1 + \alpha \cdot overlap)}$

5.5 Select $i^* = \arg\max_i \text{Score}(i)$; set $S \leftarrow S \cup \{t_{i^*}\}$, $covered \leftarrow covered \vee M[i^*]$

Phase 4: Post-processing (redundancy elimination)

6. For each $t_i \in S$ in reverse insertion order: if $\big(covered \setminus M[i]\big)$ still covers all requirements covered by S, then remove t_i from S and update $covered$ accordingly

7. Return S

Complexity

Using bitset operations, Phase 1 and 2 run in $O(nm)$, the greedy loop in $O(nm \log m)$ with efficient scanning and priority updates, and post-processing in $O(k^2 m)$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m)$ time and $O(nm)$ space, consistent with Chapter 6.

B.2. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an Intel Core i7 with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

C.3. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

C.3.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

C.3.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i, extract coverage metrics from response object M_{ij}:
   - Statement/branch/path coverage indicators
   - Mutation testing results (killed mutants, mutation score)
   - Execution outcome (pass/fail status)
   - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = \{r_1, \ldots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

C.3.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

D.4. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, October 25, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, October 25, 2025

Laith Philipp Sandouk